⚒️ Real Coder (mattock_name) - Common Errors & Fix Guidelines

📋 Change Log

Date	Change Description
Mar 18, 2026	Initial release - based on QC fail analysis from the past 4 weeks

📖 Table of Contents

1. 📘 Golden Patch / Response Quality

Issues where the Golden Patch itself fails to correctly or completely implement what the rewritten prompt requires.

Issue Type	Common Errors (Examples)	Best Practices
FAIL **Explicit Instruction Miss**	• Prompt requires npm install && npm run dev to launch everything, but the patch only installs root dependencies -	• **Read the prompt line by line** as a checklist before submitting. Every explicit

	client packages are never installed automatically. • Prompt says the app must let users filter results by person in the UI, but the feature only works via manually crafting a URL parameter. • Prompt says exit code 1 on any error, but argparse exits with code 2 for missing arguments. • Prompt requires no internet access at runtime, but the solution calls download_nltk_data() on first run. • Prompt mandates use of deepdiff or difflib, but the solution uses a custom implementation and neither library is imported. • Required file server/src/middleware/upload.ts is never created; Multer config is embedded inline instead. • Admin panel uses inline onclick handlers when the prompt explicitly bans inline scripts. • Training pipeline never calls evaluate_model() even though the prompt explicitly requires evaluation metrics to be output after training.	requirement must be traceable to code. • Search your codebase for each required feature, file, function, and constraint. If you can't find it, it's missing. • Pay special attention to startup contracts (how the app starts), error-code contracts (which HTTP/exit codes to return), and forbidden patterns (inline scripts, live fetching, banned libraries). • For frontend tasks, verify every user-facing feature is reachable through the UI, not just via the API or URL bar. • Run the rubrics against your golden patch yourself before submitting. If any criterion fails, fix the patch.
FAIL **Incorrect Code Output / Not Fulfilled**	• Unauthenticated CRUD routes return HTTP 302 (redirect) instead of the required 401. • App re-seeds the database on every startup, wiping all user-created data after restart. • Single-quote characters in plan identifiers throw a runtime error when clicking "Open."	• Run your golden patch end-to-end through all user flows, not just the happy path. • Test edge cases: single-quote inputs, first-time logins, unauthenticated requests, server restarts. • Check every explicit error code, status code, and

	• Plans panel is not shown on first login; requires a manual button click. • String-type validation for POST /api/waste is incomplete - a non-string timestamp returns 500 instead of the required 400. • Reset event flow does not propagate cleared state to other connected clients in real time; stale data remains on display/audience screens. • Test code coverage is 62% even though the prompt requires ≥80%. • Real-time vote state is stored in localStorage without a 24-hour expiration, creating a mismatch with the cookie-based enforcement.	validation rule against the prompt spec. • For real-time features (WebSockets, SSE), verify state propagation across multiple connected clients. • Run the full test suite in Docker before submitting. Never assume local success equals Docker success. • If the prompt specifies a coverage threshold, check the coverage report - not just that tests pass.
FAIL **Misleading Code Documentation**	• Fourteen source files with zero inline comments or docstrings - complex state-machine logic in admin.js is left entirely undocumented. • A README provides high-level setup but no code-level documentation anywhere in the implementation. • Non-obvious UTC timestamp arithmetic and scoring formulas have no explanation in comments. • Section-separator comments like # API are the only documentation present - no function-level explanation.	• Add docstrings to every public function and class explaining what it does, its parameters, and its return value. • Add inline comments for any non-obvious logic (complex queries, math formulas, state transitions). • A README does not substitute for code-level documentation. Both are required.

2. ✏️ Rewritten Prompt Quality

Issues with how the raw task description was translated into the structured rewritten prompt.

Issue Type	Common Errors (Examples)	Best Practices
FAIL **Prompt Conflicting Instructions**	• Prompt mandates a strict binary file layout AND requires distinct error messages for both "wrong passphrase" and "tampered/corrupted data" - physically impossible with AES-256-GCM since there is no way to distinguish the two failure modes. • Prompt requires "no internet access" during runtime, but the tech stack includes NLTK which downloads data automatically on first run. • Prompt says "runs until killed" but also adds a max_cycles parameter with a termination contract — these contradict each other.	• Before finalising the prompt, verify that **all** explicit requirements can be satisfied simultaneously by the same implementation. • Check that if the prompt specifies "no network access", then those requirements are compatible with every library in your tech stack - some (NLTK, spaCy) download data on first use. • If the original task is ambiguous, pick one interpretation and commit to it. Do not inherit contradictions from the original. • Feed the finished prompt to Cursor and validate if it follows the structure, has no ambiguity or contradictions.
NON-FAIL **Misaligned Description**	• Original task says "SQLite or JSON" but the rewrite hard-requires SQLite only, eliminating a valid solution path. • Original task mentions "a messy spreadsheet" but the rewrite adds CSV import/export and delete features that were never explicitly requested. • Title contains "(Task 9)" - an internal reference that should be stripped before submission.	• The rewrite must be a **proper detailed reconstruction** of the original task description. Every added requirement needs to be traceable to the original brief. • If the original offers flexibility (e.g., "SQLite or JSON"), preserve that flexibility. • Strip all internal references (task numbers, CB notes) from the final prompt.

	• Prompt is written in imperative style ("Build a Python CLI tool...") instead of freelance-brief style ("I need a Python CLI tool...").	

3. 🔌 Expected Interface

Issues with the Expected Interface section - the most critical part of the rewritten prompt. Every file, function, class, or API endpoint that an external test suite will interact with must be fully and accurately documented here.

Issue Type	Common Errors (Examples)	Best Practices
FAIL **Missing Interface Section / Missing Required Fields**	• The Expected Interface section is missing entirely. • Interfaces include a Name and Path but omit the required **Type**, **Input**, or **Output** fields. • evaluate_model() interface is present but the **Input** field is absent. • CLI Entry Point and SSH Connection Manager interfaces have vague inputs that do not specify parameter types. • Interfaces 3, 6, 8 are missing their **Output** field; interfaces 4, 5, 9 are missing both **Input** and **Output**. Even if a field is not applicable, it must be included and marked N/A.	• Every interface entry **must** contain all six required fields: Path, Name, Type, Input, Output, Description. No exceptions. • If a field genuinely does not apply (e.g., a static file has no Input), write N/A - do not omit the field. • Inputs must include parameter names **and** types (e.g., filepath: str, not just "a file path"). • Outputs must include the return type or HTTP response code/shape (e.g., list[tuple[str, str]], 200 OK with JSON body). • Use the interface checklist before submitting: Path ✓ Name ✓ Type ✓ Input ✓ Output ✓ Description ✓
FAIL **Undocumented Interface**	• The verifier imports app from server/index.js and sequelize, Teacher, Quiz from	• After writing your tests, go through every import statement in the test files. Every imported name from

	server/models/index.js, but neither file is documented in the Expected Interface. • The exported function getDataBase is used by tests but not documented. • The exported utility formatRelativeTime is called by external code but has no interface entry. • The public function startReminderJob is exercised by the verifier but not listed. • Frontend store members like createQuiz, stopTimer, allQuestionsAnswered are checked by tests but only a subset is documented.	your codebase must have an interface entry. • Think in terms of the test suite's perspective: what does the test need to import or call? That is your interface surface. • Helper functions and private/internal utilities do NOT need to be documented - only publicly accessible, externally called components. • Each **interface** must be **explicitly defined and non-optional.** Do not leave interfaces implicit or assume they will be inferred by the implementation or tests.
FAIL **Misleading Interface Description**	• uploadAudio is described with artwork-upload behavior from PUT /api/settings - a developer following the interface literally would build the wrong thing and fail the test. • build_photo_groups output field lists only 4 fields, but the test expects a 5th (near_duplicate_pairs) that is also in the docstring - creating a contradiction. • build_photo_groups input documents only 3 parameters, but tests call it with 7 (eps, min_samples, min_face_px, min_face_area,	• The Description field must include **everything** a test will assert. If a test checks a specific field name, DOM element, heading text, CSV header, or response shape - it must be in the Description. • After writing the interface, ask for each test: "A developer reading only this interface would know how to implement X." If false, revise. • Cross-check: run the tests against a stub implementation that follows the interface exactly. If any test fails, the interface is misleading. • List all input parameters - if a function can be called with optional keyword arguments, list them as optional (e.g., eps: float = 0.5).

	near_dup_threshold are missing). • ReadingHistory component description says "displays books in reverse chronological order" but the test requires an <h1> heading element containing "Reading History" - nothing in the description would lead a developer to add that heading. • GET /api/quizzes/:quizId/export is documented generically but the test asserts specific CSV headers (Student Name, Score, Total Questions, Completed At). • Frontend verifiers require specific DOM selectors (#title, #studentName) and text ("X out of Y") that are never mentioned in the interface description.	

4. 🧪 F2P Test Suite — Over-Specificity

The test suite must cover all explicit backend requirements without being overly specific to one particular implementation. Tests that would fail a valid alternative solution that still satisfies the prompt are a critical issue. Additionally, in the “before solution” stage, test cases should fail gracefully (e.g., via assertions) rather than erroring out due to import issues or missing files.

Issue Type	Common Errors (Examples)	Best Practices
FAIL More Than	• **Enforcing specific keyword argument style:** Tests mock	• **Test behaviour, not implementation.** Ask: "Would

5% Overly Specific Tests	watch_directory with a keyword-only signature, so any implementation calling it with positional arguments fails - even though the prompt only says "call watch_directory with the correct values." (4/38 = 10.5% - failing threshold exceeded) • **Enforcing a specific CLI flag name:** 4/24 tests hardcode --folder. The prompt only requires "accept a folder path as a command-line argument" – --directory or --path would be equally valid. • **Brittle import-path mocks:** Tests use mock.patch("src.file_sync.paramiko.SSHClient") which breaks if a developer writes from paramiko import SSHClient instead of import paramiko. • **Enforcing an idempotency contract not in the prompt:** test_sftp_client_disconnect_is_idempotent tests repeated disconnect calls - the prompt never required idempotent disconnection. • **Requiring a specific constant name:** A test asserts that a MAX_AUDIO_SIZE constant is exported. The prompt only requires enforcing a 500MB limit - how that limit is stored internally is an implementation detail. • **Hardcoding a config key path:** T61 checks app.config["DB_PATH"] directly rather than verifying the general "stores db_path in config" behaviour.	another valid solution that satisfies the prompt fail this test?" If yes, the test is overly specific. • **Avoid hardcoding specific argument names** (CLI flags, function parameter names) unless the prompt explicitly specifies them. Test that the value is accepted, not the name of the flag. • **Use interface-neutral mocks.** Patch at the library level (mock.patch("paramiko.SSHClient")) rather than patching a specific import path inside the CB's module. • **Do not test for "best practices"** the prompt didn't require (idempotency, specific constant names, type annotation conventions, specific error code naming). • Be aware that the LLMs may **introduce artificial stubs** or placeholder implementations during test setup. These can **mimic real functionality** while **masking the true implementation**, leading to misleading or **overly specific test failures** so reviewers should actively check for and remove such cases. • After writing tests, run the overly-specific audit prompt (Step 2b in the guidelines) and fix any flagged tests before proceeding. • If a requirement can only be tested in an overly specific way, cover it with a rubric criterion instead. • **Avoid hardcoding import paths** or file locations based

	• **Hardcoded import or file paths:** They are usually tied to a specific codebase structure (e.g., `codebase.app`, `codebase/index.js`). These assumptions can break valid implementations with different structures and lead to overly specific or non-portable tests.	on a specific codebase structure. **Tests should remain implementation agnostic** and verify behavior rather than enforcing a fixed file layout.
FAIL **Tests Pass on Empty Codebase (before.json has PASS)**	• Three tests designed to pass "any exception except NotImplementedError" accidentally pass on the empty codebase because it raises TypeError - causing them to show PASS in before.json. • The "before" screenshot shows tests run locally on macOS (platform: darwin), not inside Docker. The CB then changed scripts to force an all-FAIL result, producing a before.json that doesn't match the screenshot.	• **Always run the baseline (before) execution inside Docker**, never locally. The Docker environment is what QC reviewers use. • After writing tests, run them against a completely empty codebase and confirm **every single test** shows as FAILED (not ERROR, not PASSED). • For tests that catch broad exceptions, ensure the empty/stub codebase raises the specific exception being checked - or narrow the exception type to prevent accidental passes. • The before.json and after.json must be produced by the same validation script run in the same Docker environment. Never mix local runs and Docker runs. • Never modify run.sh or parsing.py "DO NOT MODIFY" sections to force a desired output.

5. 🔍 Verifier Coverage

The combination of F2P tests and rubric criteria must together cover all explicit requirements from the rewritten prompt. Gaps in coverage are one of the most common causes of QC failure.

Issue Type	Common Errors (Examples)	Best Practices
FAIL **Major** **Insufficient Verifier Coverage**	• The prompt explicitly forbids libraries (httpx, aiohttp, scrapy, click, typer, wildcard imports) but neither tests nor rubric criteria check any of these restrictions. • The prompt requires training to output evaluation metrics (ROC-AUC, F1, confusion matrix) but no test checks that train_model() actually calls evaluate_model() - tests only call it in isolation. • The prompt requires "search for keywords in the title or body" but the test only checks if a posts array is returned - it never verifies that both fields are actually searched. • Check constraints and composite unique constraints are explicit prompt requirements. Tests only validate single-column uniqueness; one test uses or True making it always pass regardless of the actual result. • Frontend key component rendering is explicitly mentioned in requirements, but there are zero frontend tests and no rubric criteria covering it. • Mobile responsiveness is a core ask in the brief but is not covered by any verifier. • W3C HTML validation and Lighthouse score ≥95 are explicit requirements with zero test or rubric coverage.	• After finalising tests and rubrics, go through every sentence of your rewritten prompt and mark each requirement as covered by a test, rubric, or both. No requirement should be left uncovered. • Use the coverage audit system prompt (Step 3b in guidelines) to check for gaps before submitting. • For banned libraries/patterns, add a rubric criterion: "The solution does not import [library X, Y, Z]." Tests cannot easily check this. • For frontend features that can be tested (component rendering, API calls, DOM assertions), write frontend tests. For features that cannot be tested automatically (layout, visual design), add rubric criteria. • Tests that only check "a response is returned" or "an array exists" are **insufficient** - they must verify the actual content/behaviour described in the prompt. • When writing a test for a function, think about its observable **side effects**, not just its return value (e.g., did it write to the database? did it call the sub-function the prompt requires?).

	• The cookie-based 24-hour voting limit is only verified to be "set" - not that it expires after 24 hours.	

6. 📊 Rubric Quality

Rubric criteria must be atomic, self-contained, positively framed, correctly weighted, and must cover all important requirements not already verified by the test suite.

Issue Type	Common Errors (Examples)	Best Practices
FAIL **Missing Criteria (Critical Requirements)**	• Rubric mostly rechecks tech-stack choices and file presence already covered by tests, while leaving important features (restart persistence, sample decision notes, working pagination, read-only shipped jobs) with no rubric coverage. • No rubric criterion checks whether the admin voting panel is only accessible when a performer's turn comes. • No criterion covers tie-breaking behaviour, admin access codes, or preventing sign-up when all slots are full. • Feature importance summary and prioritising recall for the churn class - both explicit prompt requirements - have no rubric or test coverage.	• Map every explicit requirement in the prompt to either a test or a rubric criterion (or both). If a requirement is not in your test suite, it **must** be in the rubric. • Rubrics should cover the top 30 most important requirements that **cannot** be objectively verified by unit tests - primarily UI/UX behaviour, architectural choices, and qualitative requirements. • Do not "waste" rubric slots re-verifying things the test suite already validates - use them for uncovered requirements instead. • After finalising rubrics, ask: "If the golden patch fails this criterion, does any other criterion or test catch it?" If not for an important feature, you have a coverage gap.
FAIL NON-FAIL **Overfitting /**	• **Overfitting:** "The React Router defines exactly two primary routes" - a valid solution with an additional	• Before finalising a criterion, ask: "Would a valid alternative implementation that satisfies

Underfitting Criteria	redirect or fallback route would fail, even though the prompt only requires / and /admin. • **Overfitting:** "AdminPanel displays a clear success or error message after each CRUD operation" - the prompt requires admin CRUD but never specifies a UI message requirement. • **Overfitting:** R17 requires the UI to show step numbers - the prompt only requires ascending step order, not displayed numbers. • **Overfitting:** Rubric requires chunk size to be stored as a named constant - the prompt only requires processing in fixed-size chunks. • **Overfitting:** Criterion requires type annotations in Python - the prompt never mandated type hints. • **Underfitting:** Criterion says "the app has no network access" - a solution could write locally AND make a remote API call and still pass as written.	the prompt still pass this?" If not, revise to be less specific. • Use example-based wording to avoid locking down implementation details: "uses a loop or equivalent mechanism" instead of "uses a for loop." • Criteria can mention specific examples when framed as such: "such as bcrypt or argon2" rather than "must use bcrypt." • For "no network access" criteria, phrase as an observable constraint: "The solution does not make any HTTP requests to external URLs during execution." • Check whether a criterion would fail a reasonable implementation just because it uses a different (but valid) approach than your golden patch.
NON-FAIL Overlapping / Redundant Criteria	• C#6 and C#46 both check that hyperparameters are configurable - different wording, same requirement. • C#9 and C#19 both verify that risk levels correspond to configured thresholds. • C#12 and C#21 both check that the model produces deterministic output with a fixed seed.	• Before submitting, read all rubric criteria together and look for pairs that check the same thing. If two criteria would both PASS or both FAIL on the same golden patch behaviour, merge them. • If a requirement has multiple aspects worth checking, write one criterion per aspect - but ensure the aspects are truly independent.

NON-FAIL Subjective / Vague Criteria	• "Clear separation" - between what and what? Clear to whom? • "Fragile assumption" - which assumption? How fragile? • "Meaningful comments"- what makes a comment meaningful? • "Unnecessary memory duplication" - how much is unnecessary? • "Clear, actionable messages" and "clear, descriptive" labels without defining what these terms mean in context. • "Suited for viewing across a room on a big screen" - purely subjective.	• Every criterion must evaluate to a clear "yes" or "no" without subjective interpretation. If two reviewers could disagree, make it more specific. • Replace vague qualifiers with concrete, observable characteristics: "clear error message" → "the error message includes the field name and reason for rejection." • Avoid adjectives like "appropriate," "properly," "reasonable," "best practices," and "clear" without defining what they mean in the specific context.
NON-FAIL Criteria Not Self-Contained	• "Requirements.txt includes all specified dependencies" - which dependencies? A reviewer without the prompt cannot evaluate this. • "Solution uses the specified tech stack" - which tech stack? Must be restated in the criterion. • "Functions match the naming conventions specified in the Expected Interface" - a reviewer cannot evaluate this without cross-referencing the prompt. • 3 out of 21 criteria in one submission were not self-contained - a rate of 14%, a noteworthy non-failing issue.	• Every criterion must be evaluable using only the model's code/response - without consulting the prompt, other criteria, or external information. • Embed specific details into the criterion: "requirements.txt includes Flask, SQLAlchemy, and pytest" rather than "includes all specified dependencies." • Test for self-containment: cover your prompt and try to evaluate the criterion using only the code. If you reach for the prompt, the criterion is not self-contained.
NON-FAIL Incorrect Criteria /	• C#5, C#23, C#24 check for requirements.txt version pins, snake_case, and pascalCase	• Only write criteria for things that are in the prompt. Do not add "best practice" criteria for things a good engineer would

Wrong Weights	conventions - none of these were mentioned in the prompt. • An Express error-handler criterion is given weight 5 (Mandatory) when it was never explicitly required - weight 3 would be more appropriate. • Criteria for "targeted updates" and "exposing raw database results" are marked weight 5 but are not explicitly required by the prompt.	do but the prompt never required. • **Weight 5** = you cannot imagine an acceptable response without it. **Weight 3** = substantially better with it. **Weight 1** = nice to have. Use these definitions rigorously. • If a criterion is for something you think is "good code" but the prompt didn't ask for it, either omit it or assign weight 1.

7. 🐳 Environment & Docker Setup

The Docker environment must be fully self-contained and reproducible. Any environment issue that prevents the test suite from running is a critical failure.

Issue Type	Common Errors (Examples)	Best Practices
FAIL Incompatible Environment	• Dockerfile fails to install pytest, fastapi, httpx, and other required test-suite dependencies - the container cannot run any tests. • The Dockerfile base image is changed from the required ubuntu:22.04 to python:3.11-slim, breaking the standardised environment. • Golden patch requires pillow-heic for HEIC support, but this dependency is absent from the Dockerfile — EXIF-related F2P tests fail in the container. • Dockerfile does not copy the application code into the	• Use the provided Dockerfile template without changing the base image (ubuntu:22.04) or the "DO NOT MODIFY" sections. • Add **all** required Python/system dependencies to the Dockerfile. Cross-reference every import in your codebase and test files against installed packages. • After building the Docker image, run bash run.sh **inside the container** and verify output before submitting. • If your golden patch requires a system-level dependency (e.g., libheif for HEIC), add the apt-get install command to the

	image, so the container runs tests against nothing.	SYSTEM DEPENDENCIES section. • Remember: you cannot use the COPY command in the Dockerfile. All code is injected at runtime. Only dependencies go in the Dockerfile.
NON-FAIL **run.sh Line Ending Issues**	• run.sh is saved with Windows line endings (\r\n), causing Docker to search for /bin/bash\r instead of /bin/bash. The script crashes immediately without executing any tests.	• Always save shell scripts with Unix line endings (LF, not CRLF). In VS Code, check the line-ending indicator in the bottom-right and switch to LF. • If using Windows, run dos2unix run.sh before uploading, or configure your editor to always use LF for .sh files. • After any edits to run.sh, verify it executes successfully inside the Docker container before submitting.
NON-FAIL **Python Version Mismatch**	• pyproject.toml specifies requires-python = ">=3.14" even though the rewritten prompt requires Python 3.9+ and the Docker environment uses the version installed by the template.	• Align all Python version declarations (pyproject.toml, setup.py, the prompt's tech stack, and the Dockerfile) with each other. • Default python version for the current docker file ubuntu image is 3.10, so its good practice to use python 3.10+ in the prompt if task query doesn't specify python version for better alignment • If the prompt specifies Python 3.9+, do not use syntax or packages that require Python 3.10 or higher.

💬 Frequently Asked Questions

Q1: My golden patch passes all tests locally but fails in Docker. What should I check?

The most common causes are: (1) a missing dependency in the Dockerfile - cross-check every import in your code against installed packages; (2) a system-level library present on your OS but not in Ubuntu 22.04; (3) Windows line endings in run.sh; (4) the Dockerfile base image was changed from the required ubuntu:22.04. Always run bash run.sh **inside the Docker container** before taking any screenshot or submitting.

Q2: How do I know if my interface description is "misleading"?

Imagine a developer who has read only the interface section and none of the test code. Would they know to implement every behaviour the tests will assert? If a test checks a specific HTML heading, a specific CSV column name, a specific response field, or a specific DOM selector - that detail must be in the Description. A quick check: write a stub implementation that follows only the interface documentation. If that stub fails any test, the interface is misleading.

Q3: What is the difference between "Undocumented Interface" and "Missing Interface Section"?

Missing Interface Section means the Expected Interface section is either completely absent or an existing entry is missing a required field (Path, Name, Type, Input, Output, Description). **Undocumented Interface** means the section exists and fields are correct, but a publicly accessible component that is imported or called by the test suite is simply not listed anywhere. In both cases a developer following the prompt cannot know what to build.

Q4: My test correctly fails on an empty codebase but before.json shows 3 tests as PASSED. What happened?

This usually means those tests catch a broad exception class (e.g., except Exception) and the empty codebase accidentally raises an exception of that type (like ImportError or TypeError) which passes the broad catch. Fix the test to catch a more specific exception, or import the module first so ImportError is the outer failure. Never modify the "DO NOT MODIFY" sections of run.sh or parsing.py to work around this - fix the test itself.

Q5: The prompt never mentioned type hints, but I added a rubric criterion for them. Is that okay?

No. Rubric criteria must be traceable to an explicit requirement in the rewritten prompt. If the prompt did not require type annotations, you cannot fail a response for not having them. You may add it as a weight-1 (nice-to-have) criterion **only** if it contributes

meaningfully to the evaluation and does not unfairly penalise valid implementations. When in doubt, omit it.

Q6: Can I patch `src.mymodule.SomeClass` in my tests instead of `somelib.SomeClass`?

You must be very careful here. Patching src.mymodule.SomeClass only works if the implementation imports the class that specific way. If a valid solution writes from somelib import SomeClass instead of import somelib, the patch will not intercept it and the test will fail even on a perfectly correct implementation. Prefer to patch at the source library level, or verify behaviour through observable side effects rather than mock interceptions.

Q7: My rubric has 25 criteria but some are overlapping. Is having more criteria better?

No. Overlapping criteria double-penalise the same failure without adding new coverage. If two criteria would both PASS or both FAIL for the same code behaviour, merge them. The goal is maximum **unique coverage** per criterion, not maximum count. Focus on covering 25 genuinely distinct requirements rather than restating the same requirement multiple ways.

Q8: Should I include "N/A" for Input/Output fields that don't apply (e.g., a static file)?

Yes, always. Every interface entry must include all six required fields. If Input or Output genuinely does not apply, write N/A — do not omit the field entirely. Omitting required fields is a failing issue under the "Missing Interface Section" dimension. See the flashcard example in the guidelines: even sample_deck.txt has explicit Input: N/A and Output: N/A entries.

⚠️ Final Submission Checklist

- ✓ Golden patch passes all rubric criteria when self-evaluated
- ✓ All tests FAIL on empty codebase inside Docker (not locally)
- ✓ All tests PASS with golden patch inside Docker
- ✓ Every explicit prompt requirement is covered by at least one test OR rubric criterion
- ✓ Every interface tested by the test suite is documented with all 6 required fields
- ✓ No test enforces a specific implementation detail not required by the prompt

- ✓ No rubric criterion checks something not in the prompt
- ✓ run.sh uses Unix line endings and runs inside Docker without errors
- ✓ Dockerfile uses ubuntu:22.04 as base image and includes all dependencies

Happy Tasking!! 🎉